

Лабораторная работа 4

- Выполнил: Кузьмин Артемий Андреевич
- Проверил: Наумова Надежда Александровна

Цель работы

Познакомиться с подходами к проведению нагрузочного и стресс тестирования программного ПО.

Задание

С помощью программного пакета Apache JMeter провести нагрузочное и стресс-тестирование веб-приложения в соответствии с вариантом задания.

В ходе нагрузочного тестирования необходимо протестировать 3 конфигурации аппаратного обеспечения и выбрать среди них наиболее дешёвую, удовлетворяющую требованиям по максимальному времени отклика приложения при заданной нагрузке (в соответствии с вариантом).

В ходе стресс-тестирования необходимо определить, при какой нагрузке выбранная на предыдущем шаге конфигурация перестаёт удовлетворять требованиям по максимальному времени отклика. Для этого необходимо построить график зависимости времени отклика приложения от нагрузки.

Параметры тестируемого приложения

- URL первой конфигурации (\$ 4400) - <http://stload.se.ifmo.ru:8080?token=518477805&user=-1355099300&config=1>;
- URL второй конфигурации (\$ 7200) - <http://stload.se.ifmo.ru:8080?token=518477805&user=-1355099300&config=2>;
- URL третьей конфигурации (\$ 9300) - <http://stload.se.ifmo.ru:8080?token=518477805&user=-1355099300&config=3>;
- Максимальное количество параллельных пользователей - 8;
- Средняя нагрузка, формируемая одним пользователем - 40 запросов в минуту;
- Максимально допустимое время обработки запроса - 870 мс.

Ход работы

Нагрузочное тестирование

Настроим тест план в jmeter. Сначала настроим Thread Group

Thread Group

Name:

Load Test

Comments:

Action to be taken after a Sampler error

☒ Continue

☐ Start Next Thread Loop

☐ Stop Thread

☐ Stop Test

☐ Stop Test Now

Thread Properties

Number of Threads (users):

8

Ramp-up period (seconds):

15

Loop Count:

☐ Infinite

1000

Настроим http запросы. Первая конфигурация

HTTP Request

Name:

Config 1

Comments:

Basic

Advanced

Web Server

Protocol [http]:

http

Server Name or IP:

localhost

Port Number:

8080

HTTP Request

GET

Path:

Content encoding:

☐ Redirect Automatically

☒ Follow Redirects

☒ Use KeepAlive

☐ Use multipart/form-data

☐ Browser-compatible headers

Parameters

Body Data

Files Upload

Send Parameters With the Request:

Name:	Value	URL Encode?	Content-Type	Include Equals?
token	518477805	☐	text/plain	☒
user	-1355099300	☐	text/plain	☒
config	1	☐	text/plain	☒

Вторая конфигурация

HTTP Request

Name:

Config 2

Comments:

Basic

Advanced

Web Server

Protocol [http]:

http

Server Name or IP:

localhost

Port Number:

8080

HTTP Request

GET

Path:

Content encoding:

☐ Redirect Automatically

☒ Follow Redirects

☒ Use KeepAlive

☐ Use multipart/form-data

☐ Browser-compatible headers

Parameters

Body Data

Files Upload

Send Parameters With the Request:

Name:	Value	URL Encode?	Content-Type	Include Equals?
token	518477805	☐	text/plain	☒
user	-1355099300	☐	text/plain	☒
config	2	☐	text/plain	☒

Третья конфигурация

2 / 10

HTTP Request

Name:

Config 3

Comments:

Basic

Advanced

Web Server

Protocol [http]:

http

Server Name or IP:

localhost

Port Number:

8080

HTTP Request

GET

Path:

Content encoding:

☐ Redirect Automatically

☒ Follow Redirects

☒ Use KeepAlive

☐ Use multipart/form-data

☐ Browser-compatible headers

Parameters

Body Data

Files Upload

Send Parameters With the Request:

Name:	Value	URL Encode?	Content-Type	Include Equals?
token	518477805	☐	text/plain	☒
user	-1355099300	☐	text/plain	☒
config	3	☐	text/plain	☒

Добавим Constant Throughput Timer и Duration Assertion

Так как нам нужно имитировать нагрузку в 40 запросов в минуту от одного пользователя, значение таймера поставим 320 (8 пользователей умножить на 40 запросов). Таким образом общая нагрузка от всех потоков составит $\frac{40 \text{ запр. в мин. от пользователя}}{60 \text{ с.}} \cdot 8 \text{ пользователей} = 5\frac{1}{3}$ запроса в секунду

Constant Throughput Timer

Name:

Constant Throughput Timer

Comments:

Delay before each affected sampler

Target throughput (in samples per minute):

320.0

Calculate Throughput based on:

all active threads in current thread group

Duration Assertion

Name:

Duration Assertion

Comments:

Apply to:

☐ Main sample and sub-samples

☒ Main sample only

☐ Sub-samples only

Duration to Assert

Duration in milliseconds:

870

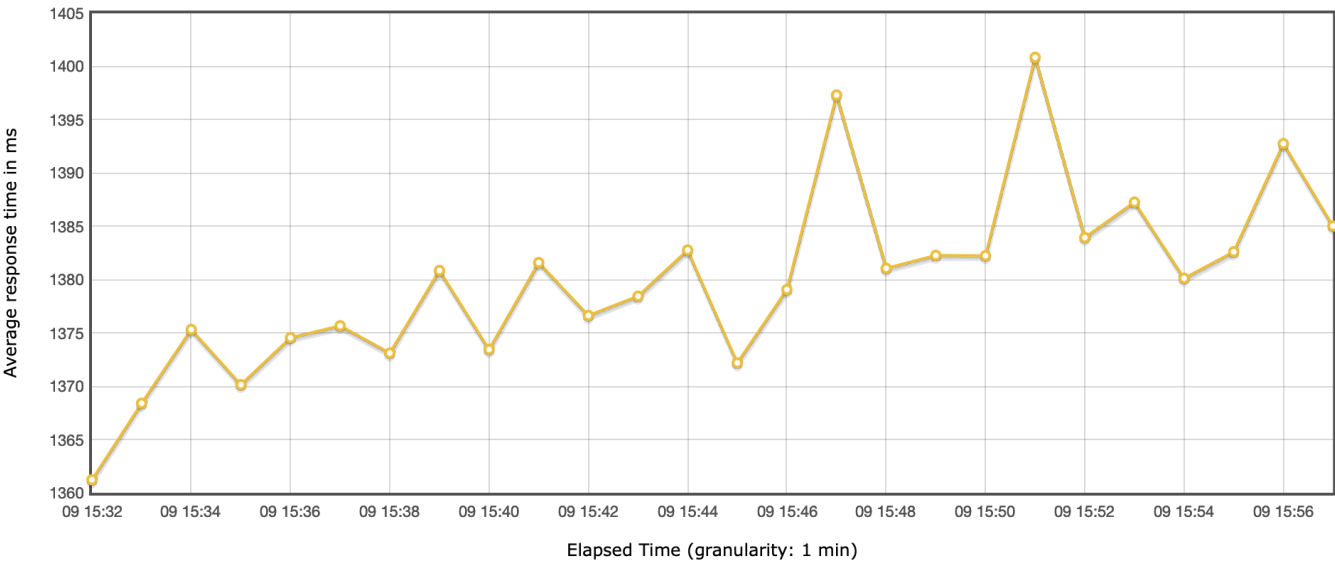
Результаты тестов

Конфигурация 1

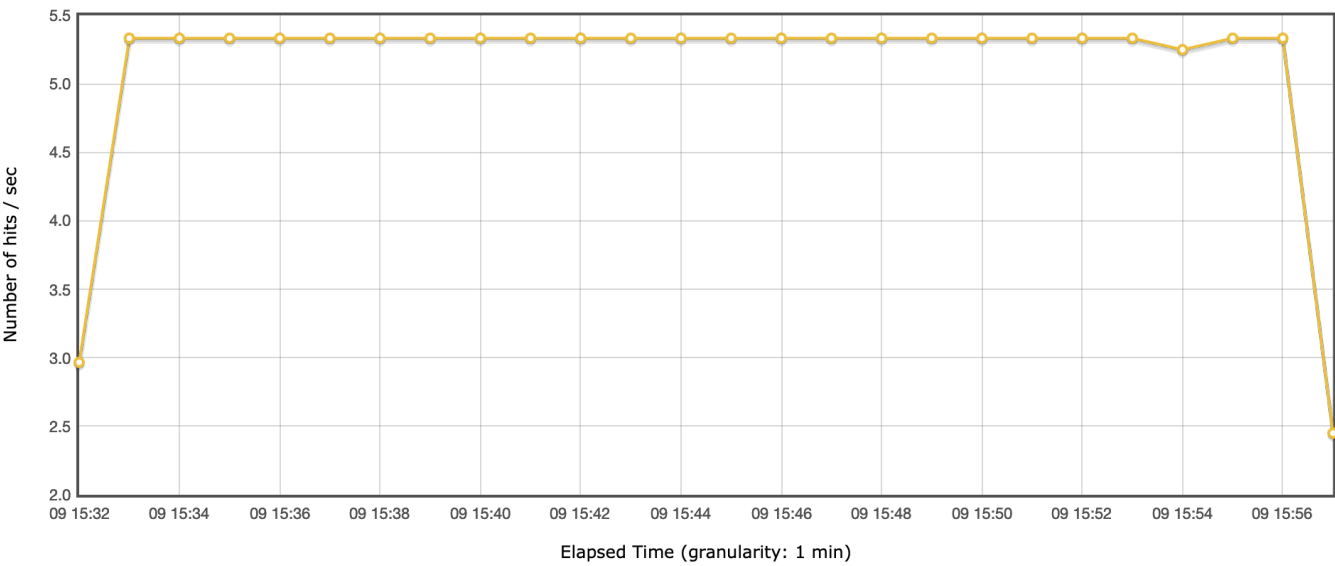
Сводная таблица

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	8000	8000	100.00%	1380.21	1310	1849	1352.00	1450.00	1471.00	1517.00	5.28	1.19	0.82
Config 1	8000	8000	100.00%	1380.21	1310	1849	1352.00	1450.00	1471.00	1517.00	5.28	1.19	0.82

Зависимость времени отклика от времени



Количество запросов в секунду



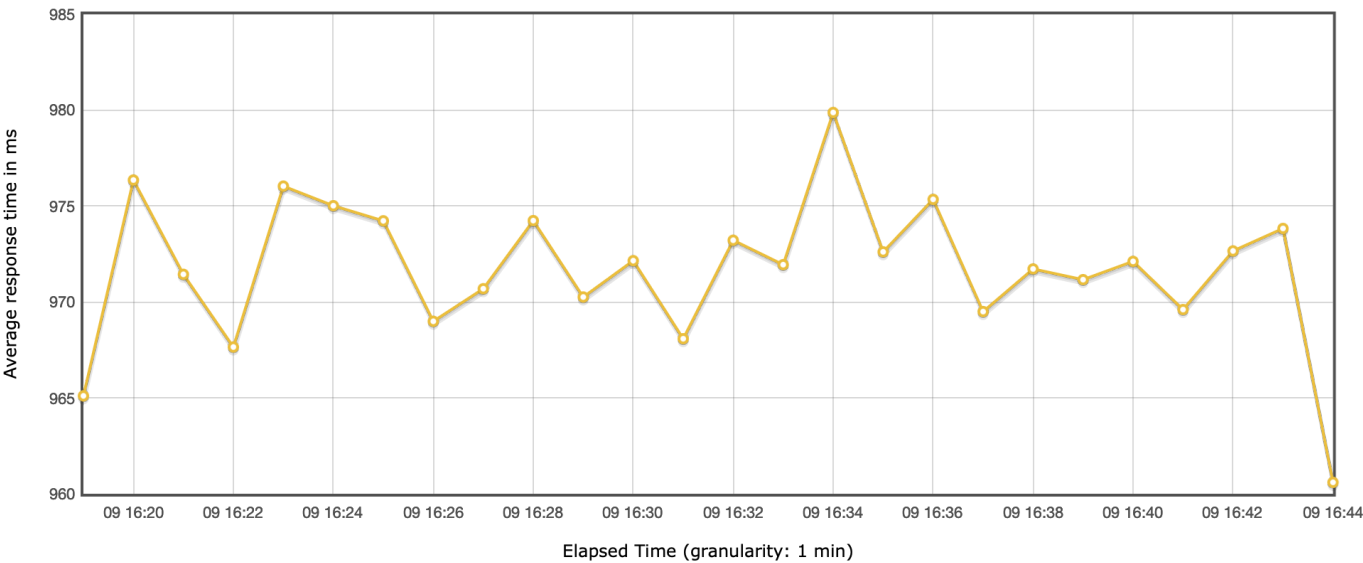
Конфигурации 1 не подходит по времени отклика, подробный отчет доступен в папке [jmeter-load-1](#)

Конфигурация 2

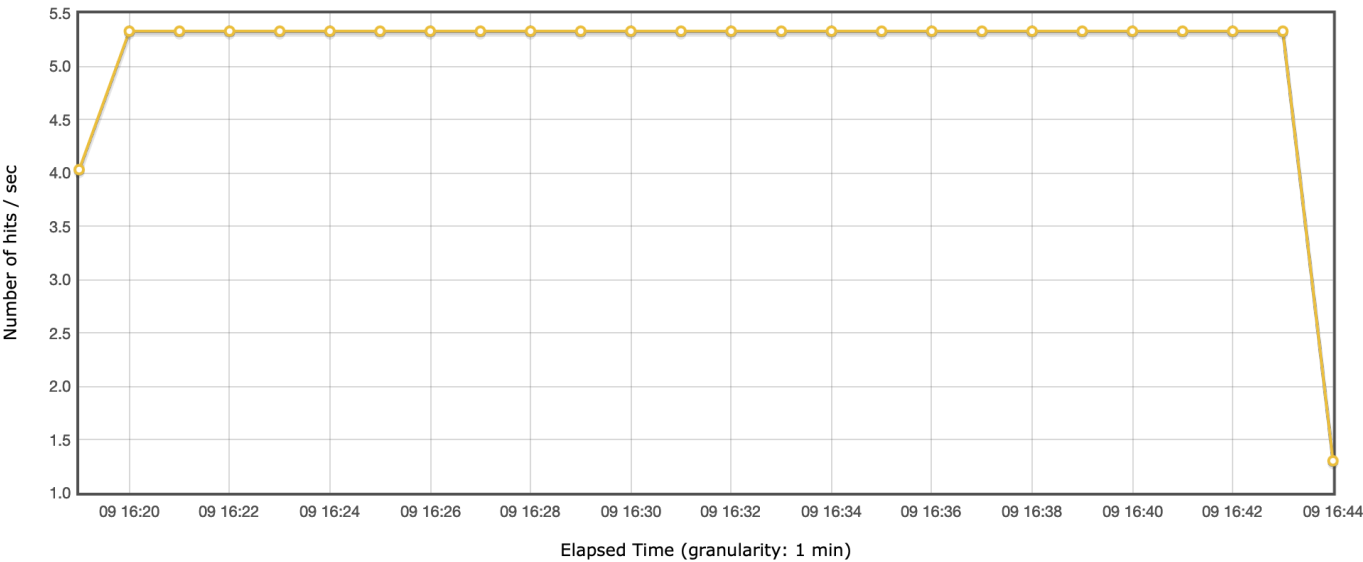
Сводная таблица

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	8000	8000	100.00%	972.11	909	1466	950.00	1049.00	1078.00	1163.00	5.30	1.20	0.82
Config 2	8000	8000	100.00%	972.11	909	1466	950.00	1049.00	1078.00	1163.00	5.30	1.20	0.82

Зависимость времени отклика от времени



Количество запросов в секунду



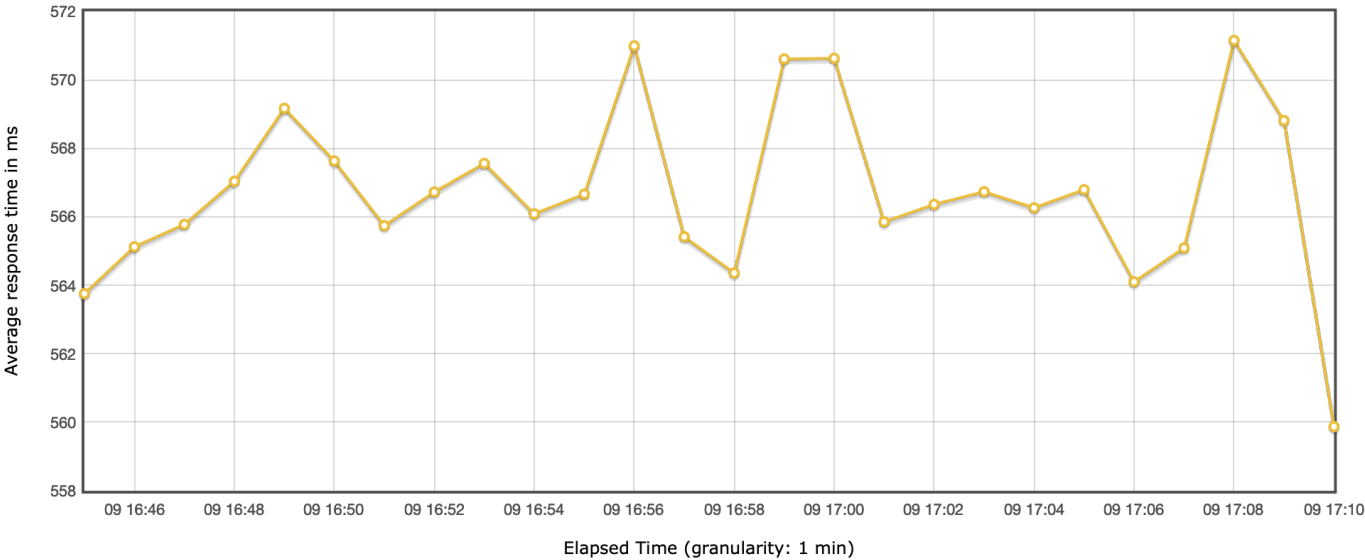
Конфигурации 2 не подходит по времени отклика, подробный отчет доступен в папке [jmeter-load-2](#)

Конфигурация 3

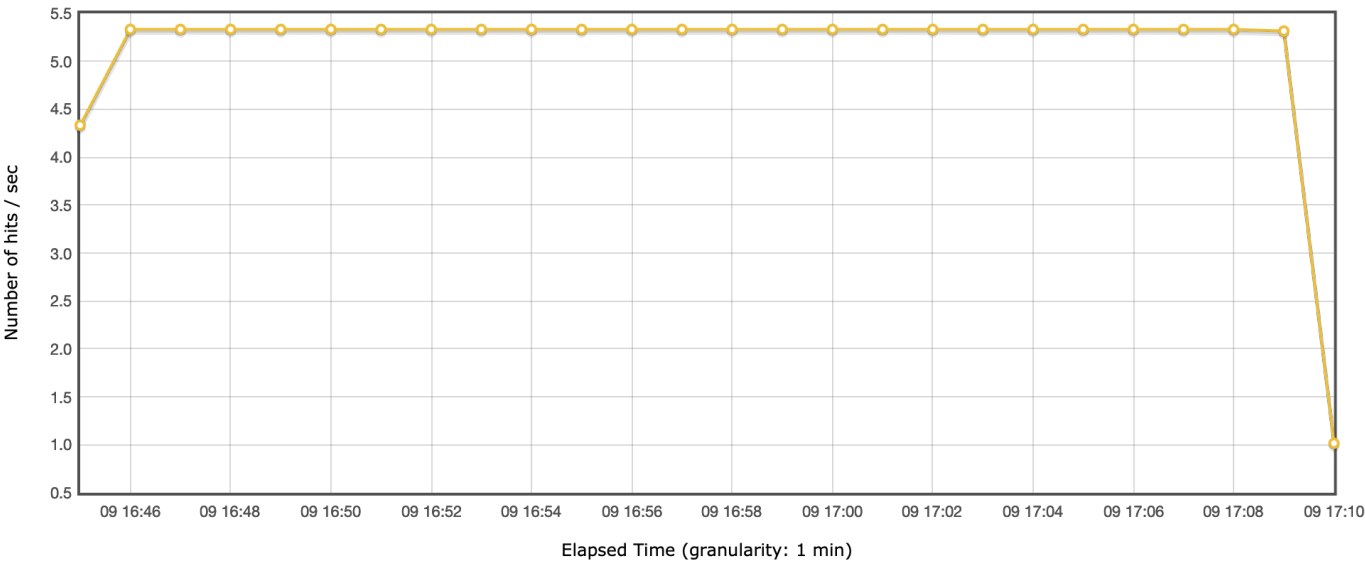
Сводная таблица

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	8000	2	0.03%	566.95	507	1021	550.00	628.00	662.00	695.00	5.32	1.20	0.83
Config 3	8000	2	0.03%	566.95	507	1021	550.00	628.00	662.00	695.00	5.32	1.20	0.83

Зависимость времени отклика от времени



Количество запросов в секунду



Конфигурации 3 подходит по времени отклика, но 2 запроса из 8000 не прошли проверку. Так как эта конфигурация показала наилучший результат, выберем ее. Подробный отчет доступен в папке [jmeter-load-3](#)

Стресс тестирование

Добавим еще одну Thread Group для стресс теста. Сначала сделаем так, что тест выполняется до первой ошибки 503. Для этого ставим loop count infinity и высокий ramp up, чтобы видеть, как меняется пропускная способность со временем

Thread Group

Name:

Comments:

Action to be taken after a Sampler error

☐ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☒ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users):

Ramp-up period (seconds):

Loop Count: ☒ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

Теперь добавим post processor, который позволит воспринимать код 500 как успех

JSR223 PostProcessor

Name:

Comments:

Script language (e.g. groovy, jexl, javascript (avoid for performances), ...)

Language: (Groovy 3.0.20 / Groovy Scripting Engine 2.0)

Parameters passed to script (exposed as 'Parameters' (type String) and 'args' (type String[]))

Parameters:

Script file (overrides script)

File Name:

Script compilation caching

Cache compiled script if available: ☒

Script (variables: ctx vars props prev sampler log Label Filename Parameters args OUT)

Script:

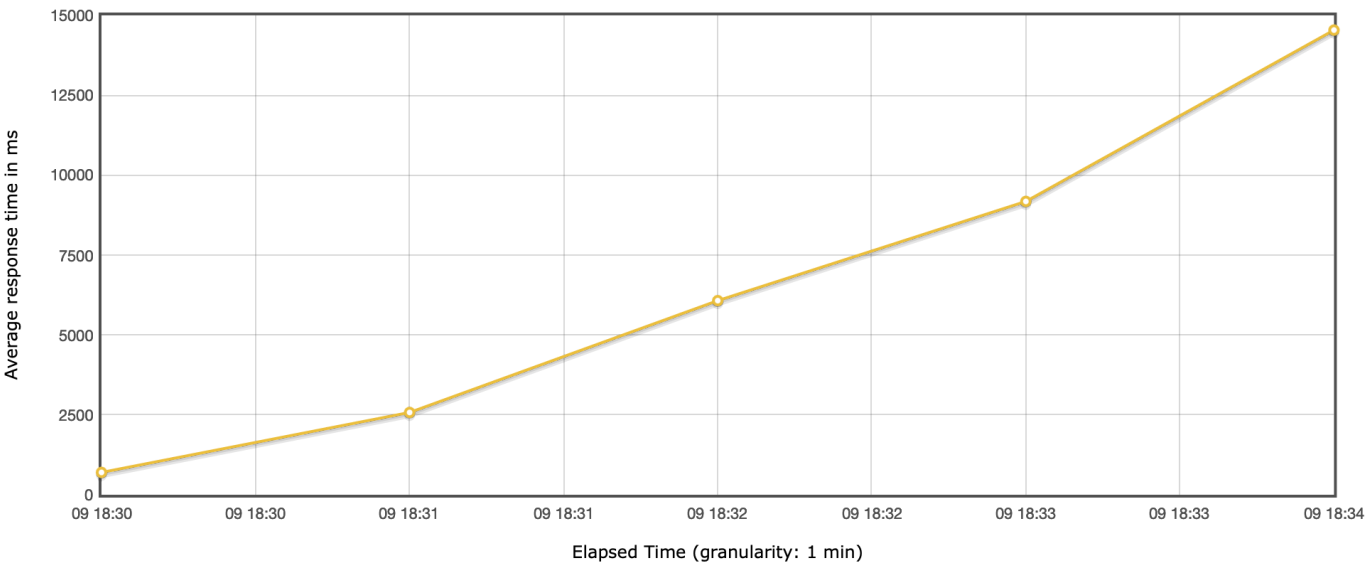
```
1 if (prev.getResponseCode() == "500") {
2     prev.setSuccessful(true);
3     prev.setResponseMessage("OK - 500 Ignore");
4 }
5
```

Конфигурация запроса аналогична прошлому этапу

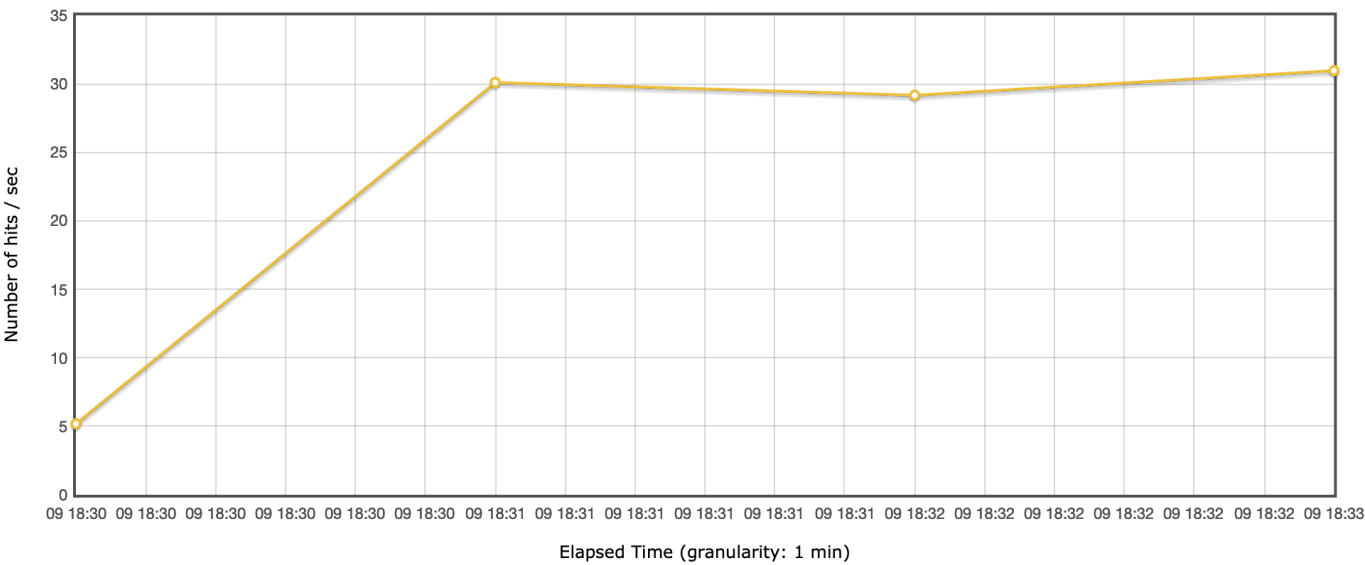
Результат теста

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	5725	28	0.49%	6165.66	153	17772	5798.00	11674.00	13192.00	17118.00	27.49	6.21	4.27
Config 3	5725	28	0.49%	6165.66	153	17772	5798.00	11674.00	13192.00	17118.00	27.49	6.21	4.27

Изменение времени отклика со временем

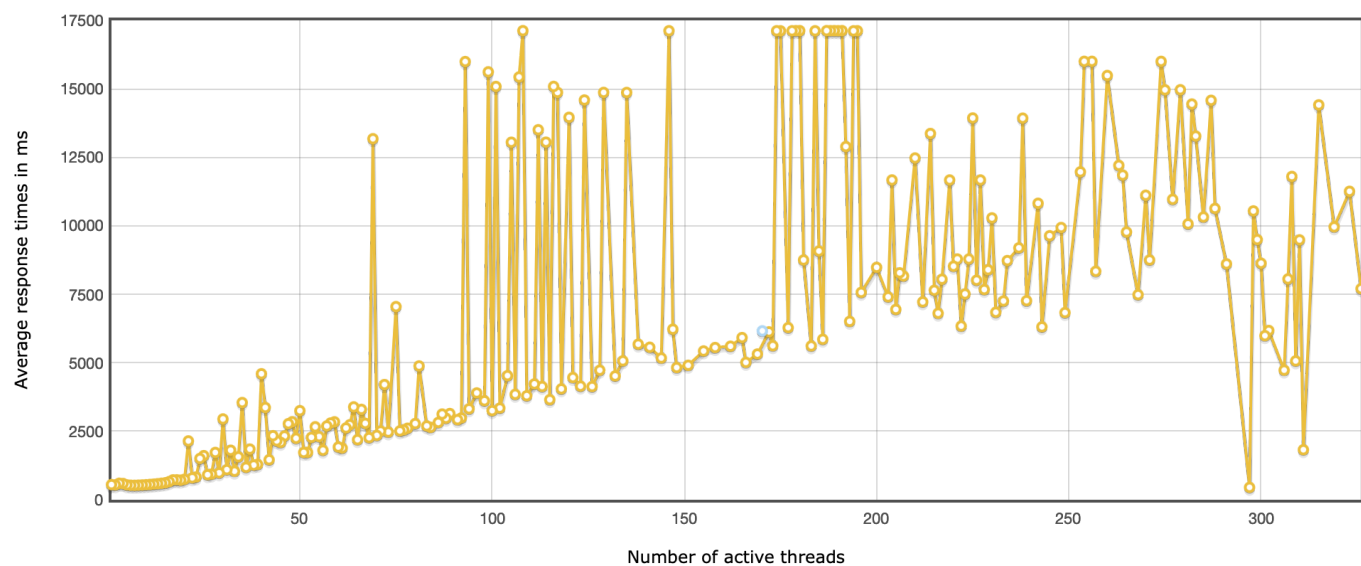


Количество запросов в секунду



По этому графику видно, что в течение нескольких минут поддерживалась пропускная способность 30 запросов в секунду, а потом сервер вернул 503

Соотношение потоков ко времени отклика



По графику можно заметить, что 503 появилась примерно при трехстах одновременных пользователях

По итогу теста имеем пиковые показатели производительности системы, после которых она перестает функционировать:

- 30 запросов в секунду
- 300 пользователей одновременно

Листинг

Исходный код доступен на [github](#)

Вывод

В ходе работы я познакомился с подходами к проведению нагрузочного и стресс тестирований, а также научился работать с jmeter